Buổi 1

Giới thiệu về Lập trình hướng đối tượng

Học phần: Lập trình hướng đối tượng

Nội dung bài học

1. Các phương pháp lập trình

1.1 Lập trình tuyến tính
1.2 Lập trình cấu trúc
1.3 Sự trừu tượng hóa dữ liệu
1.4 Lập trình hướng đối tượng là gì?

2. Các khái niệm cơ bản của OOP

2.1 Đối tượng
2.2 Lớp
2.3 Sự đóng gói
2.4 Tính kế thừa
2.5 Tính đa hình
2.6 So sánh lập trình cấu trúc và lập trình hướng đối tượng
2.7 Xem trước: các khái niệm OOP trong mã C++

3. Ngôn ngữ và ứng dụng của OOP

3.1 Các ngôn ngữ lập trình hướng đối tượng
3.2 Ứng dụng của OOP và lộ trình môn học

4. Tổng kết và thực hành

4.1 Bài thực hành buổi 1 – Phân tích bài toán theo hai cách
4.2 Câu hỏi ôn tập

1.1 Lập trình tuyến tính

- **Lập trình tuyến tính (không cấu trúc):** chương trình là **một dãy lệnh** thực hiện tuần tự từ đầu đến cuối; dùng lệnh nhảy (GOTO) để lặp và rẽ nhánh; dữ liệu là **toàn cục**, mọi lệnh đều truy cập được.

Ví dụ 1.1 – Chương trình BASIC kiểu cũ: tính tổng các số nhập vào cho đến khi gặp 0

```
10 S = 0
20 INPUT X
30 IF X = 0 THEN GOTO 60
40 S = S + X
50 GOTO 20
60 PRINT "TONG = "; S
70 END
```

- **Ưu điểm:** đơn giản, phù hợp chương trình **nhỏ** (vài chục dòng).
- **Nhược điểm:**
  - Không tái sử dụng được đoạn mã: cần tính tổng ở 3 chỗ ⇒ chép lại 3 lần.
  - Lệnh GOTO nhảy lung tung ⇒ "mã spaghetti", rất khó đọc và sửa lỗi.
  - Dữ liệu toàn cục: lệnh ở bất kỳ đâu cũng có thể làm hỏng dữ liệu.
  - Không phù hợp khi chương trình lớn (hàng nghìn dòng, nhiều người cùng viết).

Câu hỏi đặt ra

Làm sao **chia nhỏ** chương trình lớn để quản lý và dùng lại? ⇒ Lập trình **cấu trúc** (thủ tục, hàm).

1.2 Lập trình cấu trúc (lập trình thủ tục)

- **Ý tưởng:** chia chương trình thành các **chương trình con** (thủ tục, hàm), mỗi chương trình con đảm nhận một công việc; dùng các cấu trúc điều khiển tuần tự – rẽ nhánh – lặp, **không dùng GOTO**. Phương pháp thiết kế **từ trên xuống** (top-down). Ngôn ngữ tiêu biểu: Pascal, C.

Công thức của N. Wirth

Chương trình = Cấu trúc dữ liệu + Giải thuật

Ví dụ 1.2 – Quản lý sinh viên viết bằng C (lập trình cấu trúc)

```
struct SinhVien {
   char  MaSV[11];
   char  HoTen[31];
   float DTB;
};
SinhVien DS[100];  int N;      // dữ liệu
void NhapDS();                  // các hàm xử lý
void XuatDS();
float TinhDTBLop();
void SapXepTheoDTB();
```

- Dữ liệu (struct) và các hàm xử lý **tách rời** nhau.

- **Ưu điểm:** chương trình rõ ràng, dễ đọc; hàm dùng lại được; dễ chia việc cho nhiều người.
- **Nhược điểm:**
  - Dữ liệu và hàm **tách rời**: bất kỳ hàm nào cũng có thể sửa trực tiếp `DS[i].DTB = 15` (sai miền giá trị) – **không bảo vệ** được dữ liệu.
  - Khi **thay đổi cấu trúc dữ liệu** (thêm trường, đổi kiểu) phải sửa **mọi hàm** dùng đến nó.
  - Khó mô hình hóa thế giới thực (vốn gồm các **đối tượng** có cả đặc điểm lẫn hành vi).
  - Khó mở rộng: thêm loại "sinh viên cao học" có thêm trường ⇒ chép và sửa lại cả bộ hàm.

1.3 Sự trừu tượng hóa dữ liệu

- **Trừu tượng hóa:** tập trung vào **cái gì** (chức năng) mà bỏ qua **làm thế nào** (chi tiết cài đặt).

Mức trừu tượng	Ý nghĩa	Ví dụ
Trừu tượng hóa **chức năng**	Dùng hàm mà không cần biết bên trong viết thế nào	Gọi `sqrt(x)`, `printf(...)`
Trừu tượng hóa **dữ liệu**	Định nghĩa kiểu dữ liệu qua **tập giá trị + tập phép toán**, che giấu cách biểu diễn	Kiểu `float`: dùng +, −, *, / mà không quan tâm cách lưu dấu phẩy động

Kiểu dữ liệu trừu tượng (ADT – Abstract Data Type)

Một kiểu dữ liệu do người dùng định nghĩa, gồm **dữ liệu** và **các thao tác** trên dữ liệu đó; người sử dụng chỉ làm việc qua các thao tác, **không** truy cập trực tiếp dữ liệu bên trong.

Ví dụ 1.3 – ADT Phân số

- **Dữ liệu:** tử số, mẫu số (mẫu ≠ 0, luôn tối giản).
- **Thao tác:** tạo phân số, cộng, trừ, nhân, chia, so sánh, in ra.
- Người dùng viết `c = a + b` mà không cần biết bên trong quy đồng, rút gọn thế nào. Nếu đổi cách lưu (ví dụ lưu dưới dạng số thực) thì chương trình dùng phân số **không phải sửa**.
- C không hỗ trợ trực tiếp ADT (không ngăn được việc gán `a.mau = 0`); **C++ hiện thực ADT bằng lớp (class)** – nội dung buổi 4.

1.4 Lập trình hướng đối tượng là gì?

Lập trình hướng đối tượng (Object-Oriented Programming – OOP)

Phương pháp lập trình lấy **đối tượng** làm nền tảng: chương trình là tập hợp các đối tượng **tương tác** với nhau bằng cách **gửi thông điệp**. Mỗi đối tượng **gói** cả **dữ liệu** (thuộc tính) và **các hàm xử lý dữ liệu đó** (phương thức).

- **Lập trình cấu trúc:** "Chương trình làm **những việc gì**?" ⇒ chia theo **chức năng**.
- **OOP:** "Chương trình gồm **những đối tượng nào**, mỗi đối tượng **biết gì** và **làm được gì**?" ⇒ chia theo **đối tượng**.
- **Đặc điểm của OOP:**
  - Tập trung vào **dữ liệu** thay vì thủ tục.
  - Dữ liệu được **che giấu**, chỉ truy cập qua phương thức của đối tượng.
  - Đối tượng giao tiếp qua **thông điệp** (gọi phương thức).
  - Dễ **thêm** dữ liệu và hàm mới khi cần.
  - Thiết kế **từ dưới lên** (bottom-up): xây các lớp nhỏ rồi ghép lại.

Ví dụ 1.4 – Cùng bài toán, hai cách nhìn: "Hệ thống ATM rút tiền"

Lập trình cấu trúc	Lập trình hướng đối tượng
Hàm KiemTraThe()	Đối tượng **TheATM**: số thẻ, mã PIN; KiemTraPIN()
Hàm KiemTraSoDu()	Đối tượng **TaiKhoan**: số dư; RutTien(), XemSoDu()
Hàm TruTien()	Đối tượng **MayATM**: lượng tiền trong máy; NhaTien()
Hàm InBienLai()	Đối tượng **BienLai**: ngày giờ, số tiền; In()

- Trong OOP: MayATM gửi thông điệp `TaiKhoan.RutTien(500000)`; tài khoản **tự** kiểm tra số dư và tự cập nhật – không ai khác sửa được số dư trực tiếp.

2.1 Đối tượng (Object)

Đối tượng

Một **thực thể** cụ thể trong thế giới thực hoặc trong bài toán, có: **trạng thái** (state) – các **thuộc tính** (dữ liệu) với giá trị cụ thể; **hành vi** (behavior) – các **phương thức** (hàm) mà đối tượng thực hiện được; **định danh** (identity) – phân biệt với đối tượng khác.

Ví dụ 1.5 – Một vài đối tượng

Đối tượng	Thuộc tính (trạng thái)	Phương thức (hành vi)
Sinh viên "Nguyễn Văn An"	MaSV = SV01, DTB = 7.8, Lop = Tin K24	NhapDiem(), TinhDTB(), DangKyHocPhan()
Xe máy biển 75F1-3103	Hãng Honda, màu đỏ, tốc độ 40 km/h	TangToc(), Phanh(), ReTrai()
Tài khoản 0123456	Chủ TK, số dư 2.000.000 đ	NapTien(), RutTien(), XemSoDu()
Cửa sổ chương trình	Tọa độ, kích thước, tiêu đề	Mo(), Dong(), DiChuyen(), PhongTo()

- Đối tượng có thể **vật lý** (xe, người), **khái niệm** (tài khoản, đơn hàng, môn học) hoặc **phần mềm** (cửa sổ, nút bấm, tập tin).
- **Thuộc tính** trả lời câu hỏi: đối tượng **biết** gì? **Phương thức** trả lời: đối tượng **làm được** gì?
- Hai đối tượng có thể có giá trị thuộc tính **giống hệt nhau** nhưng vẫn là **hai đối tượng khác nhau** (hai SV trùng tên, cùng điểm).
- Trạng thái của đối tượng chỉ thay đổi thông qua **phương thức** của chính nó: muốn số dư tăng phải gọi NapTien(), không gán trực tiếp.

2.2 Lớp (Class) và thông điệp (Message)

Lớp

Lớp là **khuôn mẫu** (bản thiết kế) mô tả các đối tượng có **cùng thuộc tính và cùng phương thức**. Đối tượng là một **thể hiện** (instance) cụ thể của lớp.

- Lớp ↔ **kiểu dữ liệu**; đối tượng ↔ **biến** của kiểu đó.
- Một lớp tạo ra **nhiều** đối tượng; mỗi đối tượng có **giá trị thuộc tính riêng** nhưng **dùng chung** các phương thức.

Ví dụ 1.6 – Lớp SinhVien và các đối tượng

```
Lớp SinhVien
   Thuộc tính: MaSV, HoTen, DTB
   Phương thức: Nhap(), Xuat(), XepLoai()

Đối tượng sv1: (SV01, "An",   7.8)
Đối tượng sv2: (SV02, "Bình", 9.1)
```

- sv1.XepLoai() ⇒ "Khá"; sv2.XepLoai() ⇒ "Xuất sắc": cùng phương thức, kết quả khác vì **dữ liệu khác**.

Thông điệp

Thông điệp là yêu cầu gửi tới một đối tượng để nó thực hiện một **phương thức**. Gồm: đối tượng nhận, tên phương thức, các tham số.

- Cú pháp C++: `doiTuong.phuongThuc(thamSo);`

Ví dụ 1.7 – Thông điệp

```
taiKhoan.RutTien(500000);
//  ↑          ↑         ↑
// nhận   phương thức  tham số
```

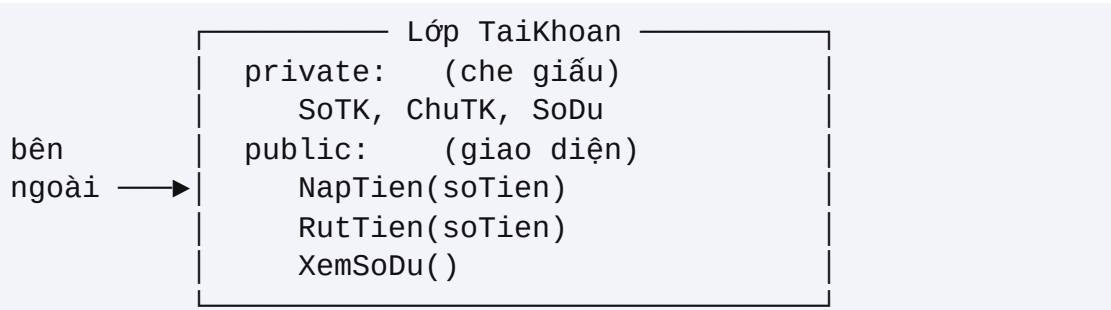
- Đối tượng nhận **tự quyết định** cách xử lý: nếu số dư không đủ, RutTien từ chối và báo lỗi.
- Người gửi **không cần biết** bên trong tài khoản lưu số dư thế nào.

- So với C: `RutTien(&tk, 500000)` – hàm đứng ngoài, dữ liệu truyền vào; trong OOP: dữ liệu và hàm **thuộc về** đối tượng.

2.3 Sự đóng gói (Encapsulation)

Đóng gói

Đóng gói là cơ chế **gom** dữ liệu và các hàm xử lý dữ liệu vào **một đơn vị** (lớp), đồng thời **che giấu thông tin** (information hiding): dữ liệu bên trong không cho bên ngoài truy cập trực tiếp mà chỉ qua các phương thức **công khai** (giao diện).



- Bên ngoài **chỉ** gọi được NapTien, RutTien, XemSoDu; **không** gán được `SoDu = -1000`.

- **Lợi ích của đóng gói:**
  - **Bảo vệ dữ liệu:** phương thức kiểm tra tính hợp lệ (số tiền rút > 0 và ≤ số dư) ⇒ đối tượng luôn ở trạng thái đúng.
  - **Giảm phụ thuộc:** thay đổi cách cài đặt bên trong (đổi SoDu từ `double` sang `long long` đồng) mà **không** ảnh hưởng chương trình bên ngoài.
  - **Dễ bảo trì, dễ gỡ lỗi:** số dư sai ⇒ chỉ cần kiểm tra vài phương thức của lớp.
  - **Dễ sử dụng:** người dùng lớp chỉ cần biết giao diện.

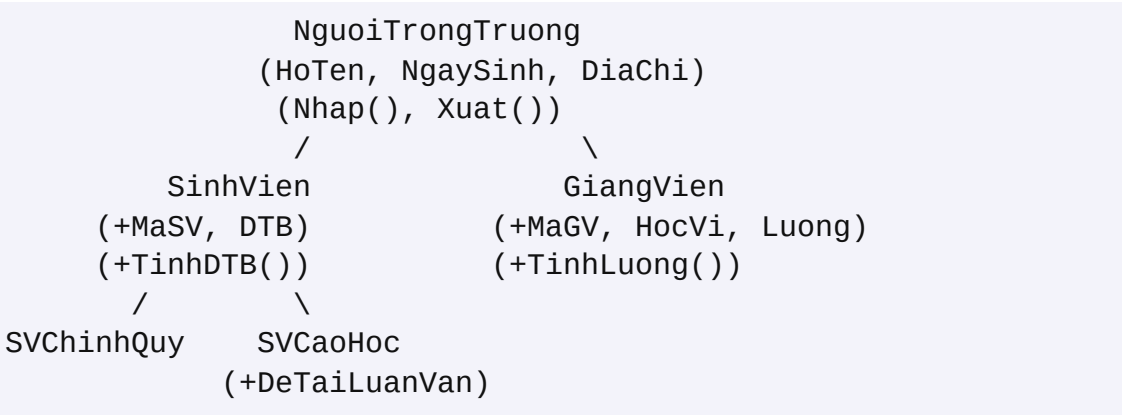
Ví dụ 1.8 – Đời thường

Lái xe chỉ cần **vô lăng, chân ga, phanh** (giao diện); không cần (và không được) can thiệp trực tiếp vào hệ thống đánh lửa hay hộp số (che giấu).

2.4 Tính kế thừa (Inheritance)

Kế thừa

Kế thừa cho phép xây dựng lớp mới (**lớp dẫn xuất**, lớp con) **từ** một lớp đã có (**lớp cơ sở**, lớp cha): lớp con **tự động có** mọi thuộc tính và phương thức của lớp cha, đồng thời **bổ sung** thành phần mới hoặc **định nghĩa lại** phương thức của cha. Quan hệ "**là một**" (is-a).



- SinhVien **là một** NguoiTrongTruong; SVCaoHoc **là một** SinhVien.

- **Lợi ích:**
  - **Tái sử dụng** mã: HoTen, NgaySinh, Nhap(), Xuat() chỉ viết **một lần** ở lớp cha.
  - **Mở rộng** dễ dàng: thêm loại "SVLienThong" chỉ cần viết phần khác biệt.
  - Tổ chức các lớp thành **cây phân cấp**, phản ánh đúng quan hệ trong thực tế.
- **Đơn kế thừa:** một lớp cha. **Đa kế thừa:** nhiều lớp cha (C++ hỗ trợ) – ví dụ TroGiang kế thừa cả SinhVien và GiangVien.

Phân biệt "là một" và "có một"

Xe **là một** phương tiện ⇒ kế thừa. Xe **có một** động cơ ⇒ **chứa** (động cơ là thuộc tính của xe), **không** kế thừa (buổi 6, 12).

2.5 Tính đa hình (Polymorphism)

Đa hình

Đa hình ("nhiều hình thái") là khả năng **cùng một thông điệp** (cùng tên phương thức) được các đối tượng **thuộc các lớp khác nhau** hiểu và thực hiện theo **cách riêng** của mình.

Ví dụ 1.9 – Thông điệp "TinhDienTich()"

Đối tượng	Cách thực hiện TinhDienTich()
HinhTron (r = 2)	$\pi \cdot r^2 = 12.57$
HinhChuNhat (3 × 4)	dài × rộng = 12
TamGiac (a = 3, b = 4, c = 5)	Công thức Heron = 6

```
Duyệt danh sách các hình:
   for mỗi hình h trong DanhSach:
       tong = tong + h.TinhDienTich()
```

- Không cần `if (loại == TRÒN) … else if (loại == CHỮ NHẬT) …`: mỗi hình **tự biết** cách tính diện tích của mình.

- **Lợi ích:**
  - Viết mã **tổng quát** cho nhiều loại đối tượng.
  - Thêm loại hình mới (HinhThoi) **không phải sửa** vòng lặp tính tổng ⇒ dễ mở rộng.
- **Các dạng đa hình trong C++:**

Dạng	Thời điểm quyết định	Buổi
Đa năng hóa hàm (overloading)	Biên dịch	3
Đa năng hóa toán tử	Biên dịch	7, 8
Template	Biên dịch	11
Phương thức ảo (virtual)	**Chạy chương trình**	10

- Ví dụ đời thường: thông điệp "kêu" – chó sủa "gâu gâu", mèo kêu "meo meo", gà gáy "ò ó o".

2.6 So sánh lập trình cấu trúc và lập trình hướng đối tượng

Tiêu chí	Lập trình cấu trúc	Lập trình hướng đối tượng
Đơn vị cơ bản	Hàm (thủ tục)	Đối tượng (lớp)
Trọng tâm	Giải thuật – "làm gì"	Dữ liệu – "ai làm, biết gì"
Quan hệ dữ liệu – hàm	Tách rời	Gói chung trong lớp
Bảo vệ dữ liệu	Kém (dữ liệu thường toàn cục / truyền tự do)	Tốt (che giấu bằng private)
Tái sử dụng	Gọi lại hàm	Kế thừa lớp, dùng lại cả lớp
Mở rộng	Thường phải sửa mã cũ	Thêm lớp mới, ít sửa mã cũ (đa hình)
Thiết kế	Từ trên xuống (top-down)	Từ dưới lên (bottom-up)
Phù hợp	Chương trình vừa và nhỏ, tính toán	Hệ thống lớn, nhiều người phát triển, giao diện đồ họa
Ngôn ngữ tiêu biểu	C, Pascal, Fortran	C++, Java, C#, Python

Ví dụ 1.10 – Tình huống thay đổi yêu cầu

Hệ thống quản lý nhân viên cần thêm loại "nhân viên thời vụ" tính lương theo giờ.

- **Cấu trúc:** sửa struct (thêm trường SoGio), sửa hàm TinhLuong (thêm nhánh `if`), sửa hàm Nhap, Xuat… – mọi chỗ có `switch (loai)` đều phải sửa ⇒ dễ sót, dễ sinh lỗi mới.
- **OOP:** thêm lớp `NVThoiVu` kế thừa `NhanVien`, định nghĩa lại `TinhLuong()`; mã quản lý danh sách **không đổi** nhờ đa hình.

2.7 Xem trước: các khái niệm OOP trong mã C++

```
#include <iostream>
using namespace std;
class TaiKhoan                      // LỚP
{  private:                         // che giấu (đóng gói)
      long SoDu;
   public:                          // giao diện
      TaiKhoan() { SoDu = 0; }
      void NapTien(long t) { if (t > 0) SoDu += t; }
      int RutTien(long t)
      {  if (t <= 0 || t > SoDu) return 0;  // từ chối
         SoDu -= t;  return 1;
      }
      long XemSoDu() { return SoDu; }
};
int main()
{  TaiKhoan tk;                     // ĐỐI TƯỢNG
   tk.NapTien(2000000);             // THÔNG ĐIỆP
   cout << tk.RutTien(500000)  << " " << tk.XemSoDu() <<
endl;
   cout << tk.RutTien(9000000) << " " << tk.XemSoDu() <<
endl;
   // tk.SoDu = -1000;              // LỖI biên dịch
}
```

- **Kết quả chạy:**

```
1 1500000
0 1500000
```

- Lần rút thứ hai (9 triệu) bị **từ chối** vì vượt số dư – đối tượng **tự bảo vệ** trạng thái của mình.
- Bỏ dấu // ở dòng cuối, trình biên dịch báo:

```
error: 'long int TaiKhoan::SoDu' is private
       within this context
```

⇒ Đóng gói được **trình biên dịch** bảo đảm, không chỉ là quy ước.

Khái niệm	Trong mã
Lớp	`class TaiKhoan`
Thuộc tính	`SoDu`
Phương thức	`NapTien`, `RutTien`, `XemSoDu`
Đối tượng	`tk`
Thông điệp	`tk.NapTien(2000000)`

- Chi tiết cú pháp lớp sẽ học ở **buổi 4, 5**.

3.1 Các ngôn ngữ lập trình hướng đối tượng

Ngôn ngữ	Năm	Đặc điểm
Simula 67	1967	Ngôn ngữ đầu tiên có khái niệm lớp, đối tượng
Smalltalk	1980	OOP "thuần": mọi thứ đều là đối tượng
C++	1983	Mở rộng C (B. Stroustrup), lai: vừa cấu trúc vừa OOP; hiệu năng cao
Object Pascal / Delphi	1986	Mở rộng Pascal
Java	1995	Thuần OOP hơn C++, chạy trên máy ảo, không con trỏ tường minh
C#	2000	Nền tảng .NET (Microsoft)
Python	1991	Đa mô hình, cú pháp ngắn gọn

- **Vì sao học OOP bằng C++?**
  - Kế thừa kiến thức C đã học (cú pháp, con trỏ, mảng).
  - Thể hiện **đầy đủ** các khái niệm: đóng gói, kế thừa (kể cả đa kế thừa), đa hình, đa năng hóa toán tử, template.
  - Cho thấy rõ cơ chế **bên dưới**: cấp phát động, constructor/destructor, sao chép đối tượng, bảng phương thức ảo.
  - Dùng rộng rãi trong game, hệ thống nhúng, phần mềm hiệu năng cao.
- Học tốt OOP với C++ ⇒ chuyển sang Java/C# rất nhanh (khái niệm giống nhau).

3.2 Ứng dụng của OOP và lộ trình môn học

- **Ứng dụng của OOP:**
  - Hệ thống **giao diện đồ họa** (cửa sổ, nút, menu là đối tượng).
  - **Hệ thống thời gian thực**, mô phỏng, trò chơi điện tử (nhân vật, vật thể).
  - **Hệ quản trị CSDL hướng đối tượng**, các hệ thống thông tin quản lý lớn.
  - **Trí tuệ nhân tạo**, hệ chuyên gia; CAD/CAM; hệ thống web và di động.
  - Thư viện, framework tái sử dụng (STL, Qt, .NET).

Buổi	Nội dung
1	Giới thiệu OOP
2, 3	Các mở rộng của C++ so với C
4, 5, 6	Lớp, đối tượng, constructor/destructor, thành viên đặc biệt
7, 8	Đa năng hóa toán tử, sao chép sâu
9	Kế thừa
10	Đa hình
11	Template, nhập/xuất
12	Thiết kế chương trình OOP, mini-project

Ví dụ 1.11 – Bốn trụ cột của OOP trong một câu

"Lớp **Hinh đóng gói** tọa độ và màu; **HinhTron**, **HinhChuNhat kế thừa** từ Hinh; danh sách các Hinh gọi `Ve()` thì mỗi hình vẽ theo cách riêng – **đa hình**; người dùng chỉ biết `Ve()`, `DiChuyen()` mà không biết cách cài đặt – **trừu tượng hóa**."

4.1 Bài thực hành buổi 1 – Phân tích bài toán theo hai cách

Đề bài: Quản lý thư viện

Thư viện có nhiều đầu sách (mã, tên, tác giả, số lượng). Bạn đọc (mã thẻ, họ tên, hạn thẻ) được mượn tối đa 5 cuốn. Mỗi phiếu mượn ghi bạn đọc, sách, ngày mượn, ngày trả. Thủ thư lập phiếu, nhận trả, tính tiền phạt quá hạn (5.000 đ/ngày).

- **Yêu cầu 1 – theo lập trình cấu trúc:**
  - Khai báo các struct dữ liệu.
  - Liệt kê các hàm (tên, tham số, công dụng): NhapSach, TimSach, LapPhieuMuon, TraSach, TinhTienPhat…
  - Chỉ ra **3 chỗ** dữ liệu có thể bị sửa sai (ví dụ số lượng sách âm).
- **Yêu cầu 2 – theo hướng đối tượng:**
  - Xác định các **lớp** (danh từ trong đặc tả): Sach, BanDoc, PhieuMuon, ThuVien.
  - Với mỗi lớp: thuộc tính, phương thức, cái gì để **private**.
  - Xác định quan hệ: ThuVien **có** nhiều Sach, BanDoc; PhieuMuon **tham chiếu** BanDoc và Sach.
- **Yêu cầu 3 – mở rộng:** thư viện có thêm loại "tạp chí" (có số kỳ, chỉ đọc tại chỗ) và "sách tham khảo" (mượn tối đa 3 ngày). Trong mỗi cách tiếp cận, phải **sửa những gì**?

Gợi ý trả lời yêu cầu 3 (OOP)

Tạo lớp cha **TaiLieu** (mã, tên, số lượng, `ChoMuon()`), các lớp con **Sach**, **TapChi**, **SachThamKhao** định nghĩa lại `ChoMuon()` và `SoNgayMuonToiDa()`. Lớp PhieuMuon làm việc với TaiLieu chung ⇒ **không phải sửa**.

- **Sản phẩm nộp:** bảng so sánh hai cách thiết kế (1 trang).

4.2 Câu hỏi ôn tập

Câu hỏi

- 1) Nêu ưu, nhược điểm của lập trình tuyến tính, lập trình cấu trúc. Vì sao cần lập trình hướng đối tượng?
- 2) Phân biệt **lớp** và **đối tượng**; **thuộc tính** và **phương thức**. Cho 3 ví dụ.
- 3) Đóng gói là gì? Nêu 3 lợi ích. Che giấu thông tin khác gì với "không cho dùng"?
- 4) Phân biệt quan hệ "là một" (kế thừa) và "có một" (chứa). Cho ví dụ mỗi loại.
- 5) Đa hình là gì? Cho một ví dụ đa hình trong đời thường và một ví dụ trong phần mềm.

Bài tập (có gợi ý)

- 1) Xác định lớp, thuộc tính, phương thức cho: (a) hệ thống bán vé xem phim; (b) trò chơi cờ caro; (c) quản lý điểm sinh viên.
  → **Gợi ý (b):** BanCo (kích thước, mảng ô; DanhDau(), KiemTraThang()), NguoiChoi (tên, ký hiệu X/O; ChonNuocDi()), TroChoi (bàn cờ, 2 người chơi, lượt; BatDau(), DoiLuot()).
- 2) Vẽ cây kế thừa cho các phương tiện giao thông: xe đạp, xe máy, ô tô con, xe tải, tàu hỏa, máy bay. Xác định thuộc tính/phương thức ở lớp cha.
- 3) Cho thông điệp `PhatAm()` gửi tới các đối tượng nhạc cụ (đàn piano, trống, sáo). Giải thích đây là thể hiện của tính chất nào và lợi ích khi thêm nhạc cụ mới.